



# Low-Cost Real-Time ECG Monitoring System with Artificial Intelligence-Assisted Arrhythmia Analysis

Guilherme Martins Specht

Polytech Montpellier

University of Montpellier

2026

*Projet de Fin d'Études Report*

# Contents

# Preface

This report presents the development of an intelligent real-time electrocardiogram monitoring system based on embedded electronics, digital signal processing, and artificial intelligence.

The project was developed as a proof-of-concept biomedical engineering system. Its objective was not to produce a clinically certified medical device, but to demonstrate the feasibility of acquiring ECG signals using low-cost hardware, processing them in real time, detecting cardiac rhythm parameters, and applying a Convolutional Neural Network for preliminary rhythm classification.

Some limitations were encountered during development, especially related to signal quality, electrode contact, noise, and the integration between embedded firmware and the Python-based monitoring interface. These limitations are discussed throughout the report because they are important for understanding the practical constraints of low-cost ECG acquisition systems.

# 1 Introduction

Cardiovascular diseases are among the leading causes of death worldwide and represent a major challenge for modern healthcare systems. Early detection of cardiac abnormalities is essential for reducing mortality rates and improving patient outcomes. Electrocardiography (ECG) is one of the most important non-invasive diagnostic tools used to evaluate the electrical activity of the heart.

Despite the effectiveness of ECG analysis, continuous interpretation of cardiac signals often requires specialized professionals and expensive medical equipment. In emergency situations, delays in ECG interpretation may compromise rapid clinical decision-making. Therefore, low-cost intelligent monitoring systems capable of processing ECG signals in real time are becoming increasingly relevant.

Artificial Intelligence (AI), particularly Deep Learning methods such as Convolutional Neural Networks (CNNs), has demonstrated promising performance in biomedical signal classification tasks. By combining embedded systems with AI algorithms, it becomes possible to create portable and accessible devices capable of assisting in cardiac monitoring.

This work proposes the development of a low-cost intelligent ECG monitoring system based on the ESP32-S3 microcontroller, AD8232 analog front-end, ADS1115 analog-to-digital converter, and a real-time AI-based classification system implemented using Python and CNN models.

## 1.1 Contextualization

Cardiovascular diseases frequently require continuous monitoring for diagnosis, follow-up, and emergency response. Traditional ECG devices are reliable, but they are often expensive, less portable, and dependent on clinical infrastructure.

Portable cardiac monitoring systems allow continuous acquisition of ECG signals outside hospital environments. The growing need for intelligent healthcare devices has encouraged the development of embedded systems capable of real-time signal acquisition, processing, and analysis.

## 1.2 Research Problem

The main research question addressed in this work is:

> How can a low-cost embedded system capable of acquiring, processing, and classifying ECG signals in real time using artificial intelligence be developed?

## 1.3 Objectives

### 1.3.1 General Objective

To develop an intelligent real-time ECG monitoring system using embedded electronics, digital signal processing, and artificial intelligence.

## 1.4 Cardiac Heartbeat Dynamics

The heartbeat is generated by the electrical conduction system of the heart. The sinoatrial node (SA node), located in the right atrium, acts as the natural pacemaker and initiates the electrical impulse responsible for cardiac contraction.

The electrical activity propagates through the atria, producing the P wave in the ECG signal. After passing through the atrioventricular node (AV node), the impulse propagates through the ventricles, generating the QRS complex, which represents ventricular depolarization. Finally, ventricular repolarization produces the T wave.

A complete cardiac cycle corresponds to one heartbeat and is composed of two main mechanical phases:

- Systole: contraction phase responsible for blood ejection;
- Diastole: relaxation phase responsible for ventricular filling.

The ECG waveform acquired in this project is mainly characterized by the detection of the QRS complex because it presents the highest amplitude and provides reliable heartbeat timing information.

The electrical activity of the heart can be summarized as:

$$ECG(t) = P(t) + QRS(t) + T(t) \tag{1}$$

where:

- $P(t)$ represents atrial depolarization;
- $QRS(t)$ represents ventricular depolarization;
- $T(t)$ represents ventricular repolarization.

The heart rate is generally measured in beats per minute (BPM). In healthy adults at rest, the normal heart rate usually ranges between:

$$60 \leq BPM \leq 100 \tag{2}$$

Bradycardia occurs when:

$$BPM < 60 \tag{3}$$

Tachycardia occurs when:

$$BPM > 100 \tag{4}$$

where $RR$ represents the time interval between two consecutive R-peaks.

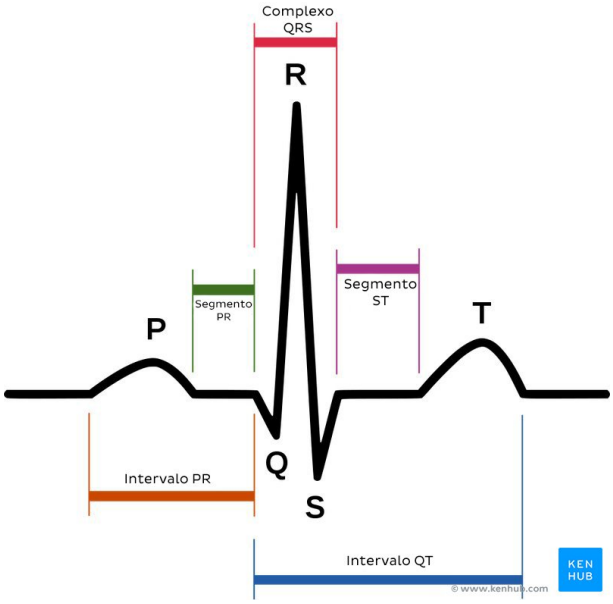


Figure 1: Typical ECG waveform showing P wave, QRS complex, and T wave.

### 1.4.1 Specific Objectives

- Acquire ECG signals using the AD8232 module;
- Convert the analog ECG signal using the ADS1115 ADC;
- Process ECG signals using digital signal processing techniques;
- Detect R-peaks using a Pan-Tompkins-inspired algorithm;
- Calculate BPM and RR intervals;
- Classify ECG windows using a Convolutional Neural Network;
- Develop a real-time graphical interface for monitoring and analysis.

The heart rate in beats per minute is calculated according to:

$$BPM = \frac{60}{RR} \tag{5}$$

where $RR$ is the interval between two consecutive R-peaks in seconds.
The RR interval is obtained by:

$$RR = t_{R_n} - t_{R_{n-1}} \tag{6}$$

where $t_{R_n}$ is the timestamp of the current R-peak and $t_{R_{n-1}}$ is the timestamp of the previous R-peak.

## 1.5 Justification

The proposed system has clinical and engineering relevance due to its low cost, portability, and ability to perform real-time ECG analysis. The integration of embedded systems and AI may contribute to faster cardiac assessment, especially in educational environments, biomedical research, and preliminary monitoring applications.

The project also has strategic relevance because it combines several engineering areas: analog signal acquisition, embedded firmware, digital signal processing, machine learning, and human-machine interface development.

# 2 Development of the Intelligent ECG Monitoring System

## 2.1 Embedded System State Machine

The firmware implemented in the ESP32-S3 follows a finite state machine architecture to organize the ECG acquisition, processing, and visualization pipeline. This approach improves modularity, timing determinism, debugging, and scalability.

The system operation is divided into several states, each responsible for a specific task in the monitoring pipeline.

The main states are:

- **BOOT**: hardware initialization and peripheral configuration;
- **IDLE**: waiting for valid ECG signal acquisition;
- **ACQUIRE**: ECG signal acquisition using the ADS1115 converter;
- **FILTER**: preprocessing and digital filtering;
- **DETECT**: R-peak and BPM calculation;
- **CLASSIFY**: CNN inference and arrhythmia prediction;
- **DISPLAY**: OLED update and serial communication;
- **ERROR**: signal acquisition or processing failure handling.

The firmware transitions between states according to timing conditions, signal quality, and processing results.

The global execution flow can be summarized as:

$$\text{BOOT} \rightarrow \text{IDLE} \rightarrow \text{ACQUIRE} \rightarrow \text{FILTER} \rightarrow \text{DETECT} \rightarrow \text{CLASSIFY} \rightarrow \text{DISPLAY}$$

If an abnormal condition occurs:

$$\text{ANY STATE} \rightarrow \text{ERROR} \rightarrow \text{IDLE}$$

During the BOOT state, all peripherals are initialized, including:

- I2C communication;

- ADS1115 configuration;
- OLED initialization;
- serial communication setup;
- ECG buffers allocation.

In the ACQUIRE state, ECG samples are continuously obtained from the ADS1115 analog-to-digital converter.

In the FILTER state, the acquired ECG signal undergoes preprocessing operations such as smoothing, derivative calculation, squaring, and moving window integration.

In the DETECT state, the algorithm searches for valid R-peaks while enforcing a refractory period to avoid false detections.

The CLASSIFY state is responsible for organizing ECG windows and sending them to the CNN inference module running in Python.

Finally, the DISPLAY state updates:

- BPM value;
- RR interval;
- signal quality indicator;
- ECG waveform;
- CNN prediction;
- confidence estimation.

The ERROR state is triggered whenever the ECG signal becomes unstable, disconnected, excessively noisy, or invalid for analysis.

This finite state machine architecture improves reliability and simplifies future firmware expansion, especially for additional biomedical signal processing tasks and TinyML deployment directly on the ESP32-S3.

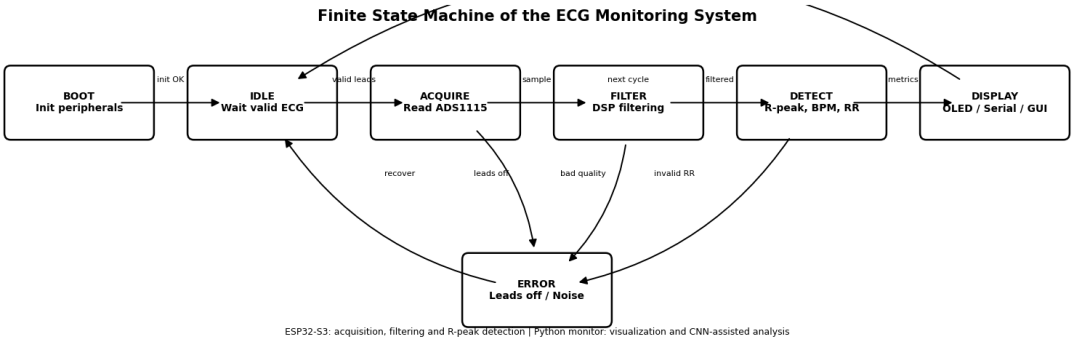


Figure 2: Finite state machine used in the embedded ECG monitoring system.

## 2.2 General System Architecture

The proposed system is composed of five main blocks:

- ECG acquisition using electrodes and the AD8232 analog front-end;
- Analog-to-digital conversion using the ADS1115 converter;
- Embedded processing using the ESP32-S3;
- Serial communication between the embedded system and the computer;
- Real-time visualization and CNN inference using a Python monitor.

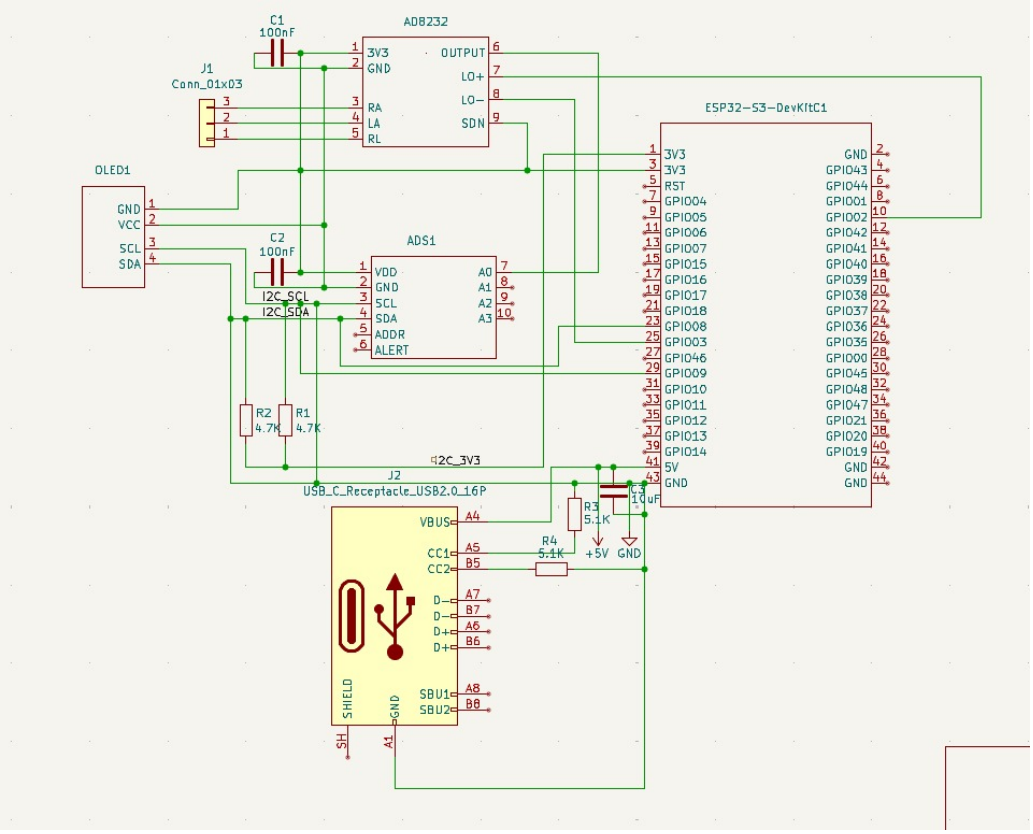


Figure 3: Circuit of the proposed ECG monitoring system.

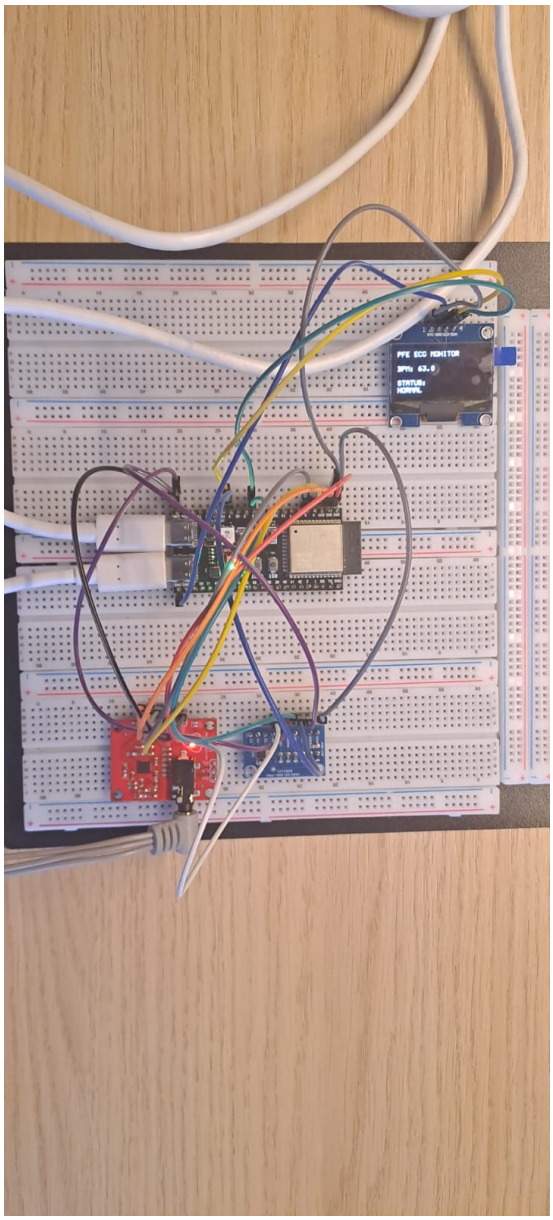


Figure 4: Real prototype mounted on a breadboard.

## 2.3 Hardware Components

### 2.3.1 ESP32-S3

The ESP32-S3 microcontroller was selected due to its processing capability, low cost, USB communication support, and compatibility with embedded real-time applications.

The ESP32-S3 is responsible for:

- Reading the converted ECG signal;
- Applying preprocessing and R-peak detection;
- Calculating BPM and RR intervals;
- Sending data to the computer through serial communication;

- Updating the OLED display.

Figure 5: ESP32-S3 development board used in the prototype.

### 2.3.2 AD8232 ECG Front-End

The AD8232 module is an analog front-end designed for ECG and biopotential signal acquisition. In this project, it was used to acquire the electrical activity of the heart through three electrodes.

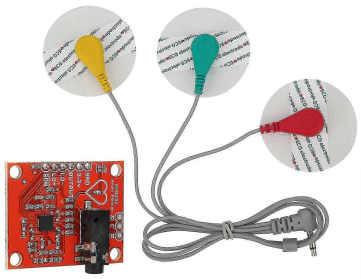

Figure 6: AD8232 ECG acquisition module.

### 2.3.3 ADS1115 Analog-to-Digital Converter

The ADS1115 is a 16-bit analog-to-digital converter used to improve ECG acquisition resolution compared with the internal ADC of the microcontroller.

The ADC resolution is given by:

$$Resolution = \frac{V_{REF}}{2^{16}} \tag{7}$$

where $V_{REF}$ is the reference voltage of the converter.

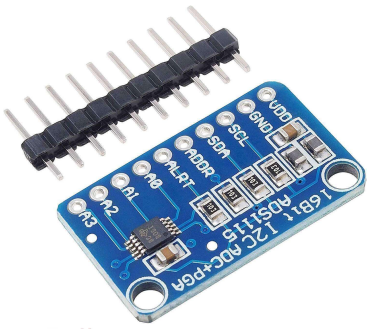


Figure 7: ADS1115 analog-to-digital converter.

## 2.4 ECG Signal Acquisition

Three electrodes were used for ECG acquisition:

- Right Arm (RA);
- Left Arm (LA);
- Right Leg (RL).

The sampling frequency adopted in this project was approximately:

$$f_s = 250\ Hz \tag{8}$$

This sampling rate is sufficient for basic ECG monitoring and R-peak detection.

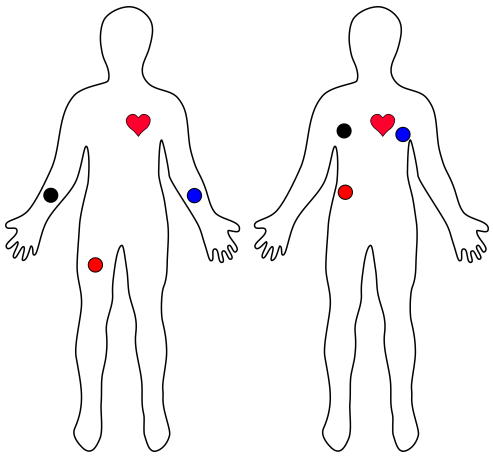

Figure 8: Electrode positioning used for ECG acquisition.

## 2.5 Digital Signal Processing

ECG signals are susceptible to different types of noise, such as baseline wander, motion artifacts, electrode displacement, and electromagnetic interference. Therefore, digital preprocessing is necessary before heartbeat detection and AI inference.

The ECG signal can be represented as:

$$ECG(t) = P(t) + QRS(t) + T(t) + n(t) \tag{9}$$

where $n(t)$ represents noise and interference.

A first-order low-pass smoothing filter may be represented as:

$$y[n] = \alpha y[n-1] + (1-\alpha)x[n] \tag{10}$$

where $x[n]$ is the input signal, $y[n]$ is the filtered output, and $\alpha$ is the smoothing coefficient.

The derivative stage emphasizes rapid variations associated with the QRS complex:

$$d[n] = x[n] - x[n-1] \tag{11}$$

The squared signal is given by:

$$s[n] = d[n]^2 \tag{12}$$

The Moving Window Integration is defined as:

$$MWI[n] = \frac{1}{N}\sum_{i=0}^{N-1} s[n-i] \tag{13}$$

where $N$ is the integration window length.

## 2.6 R-Peak Detection

R-peaks were detected using a Pan-Tompkins-inspired approach. The main processing steps were:

1. Signal filtering;
2. Derivative calculation;
3. Squaring;
4. Moving Window Integration;
5. Adaptive thresholding;
6. Refractory period validation.

The adaptive threshold is represented as:

$$TH = NPKI + 0.25(SPKI - NPKI) \tag{14}$$

where $SPKI$ is the signal peak estimate and $NPKI$ is the noise peak estimate.

A refractory period was implemented to avoid multiple detections of the same heartbeat:

$$T_{ref} > 200\ ms \tag{15}$$

## 2.7 Artificial Intelligence Methodology

### 2.7.1 Dataset

The CNN was evaluated using ECG windows extracted from a dataset composed of:

- MIT-BIH Arrhythmia Database samples;
- Synthetic ECG signals generated for controlled experiments;
- Real ECG captures obtained using the developed prototype.

The final dataset used for the CNN analysis contained:

$$N_{total} = 23005 \tag{16}$$

ECG windows. From this dataset, 20% was used as the test set:

$$N_{test} = 4601 \tag{17}$$

The evaluated classes were:

- Fusion;
- Normal;
- Other;
- Supraventricular;
- Ventricular.

### 2.7.2 Preprocessing

Before being applied to the CNN, the ECG windows were normalized according to:

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{18}$$

where $\mu$ is the mean value and $\sigma$ is the standard deviation of the signal window.

### 2.7.3 CNN Architecture

The CNN architecture was based on one-dimensional convolutional layers. This structure is suitable for ECG analysis because it can extract temporal patterns from sequential biomedical signals.

A one-dimensional convolution operation is defined as:

$$y[n] = \sum_{k=0}^{K-1} x[n-k]h[k] \tag{19}$$

where $x[n]$ is the input signal and $h[k]$ is the convolution kernel.

The final layer uses the softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}} \tag{20}$$

where $P(y_i)$ is the probability associated with class $i$.

# 3 Graphical Interface

A Python-based graphical interface was developed for real-time monitoring. Two modes were implemented:

- A medical terminal monitor focused on BPM, RR interval, signal quality, and serial logs;
- A real-time AI monitor focused on ECG waveform visualization and CNN inference.

The interface displays:

- Real-time ECG waveform;
- BPM estimation;
- RR interval;
- Signal quality status;
- CNN classification;
- Prediction confidence;
- Serial communication terminal.

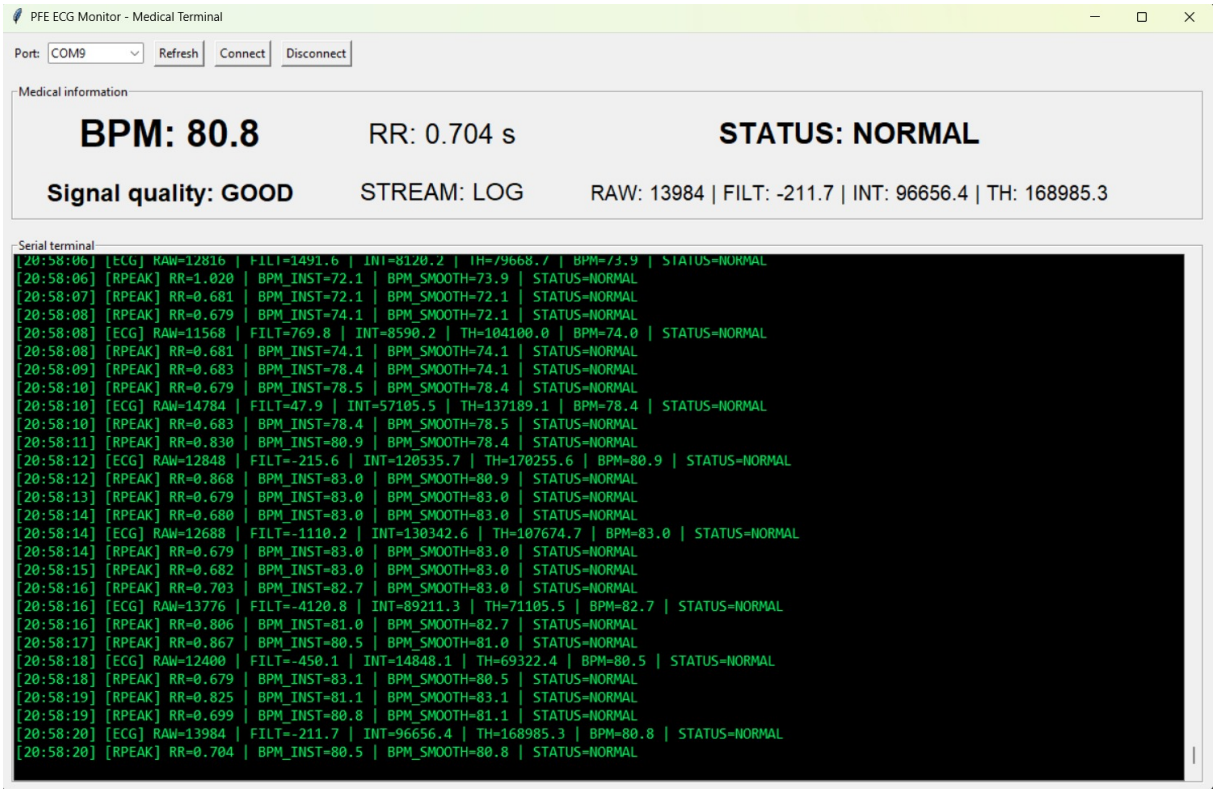


Figure 9: Medical terminal interface showing ECG parameters and serial data.

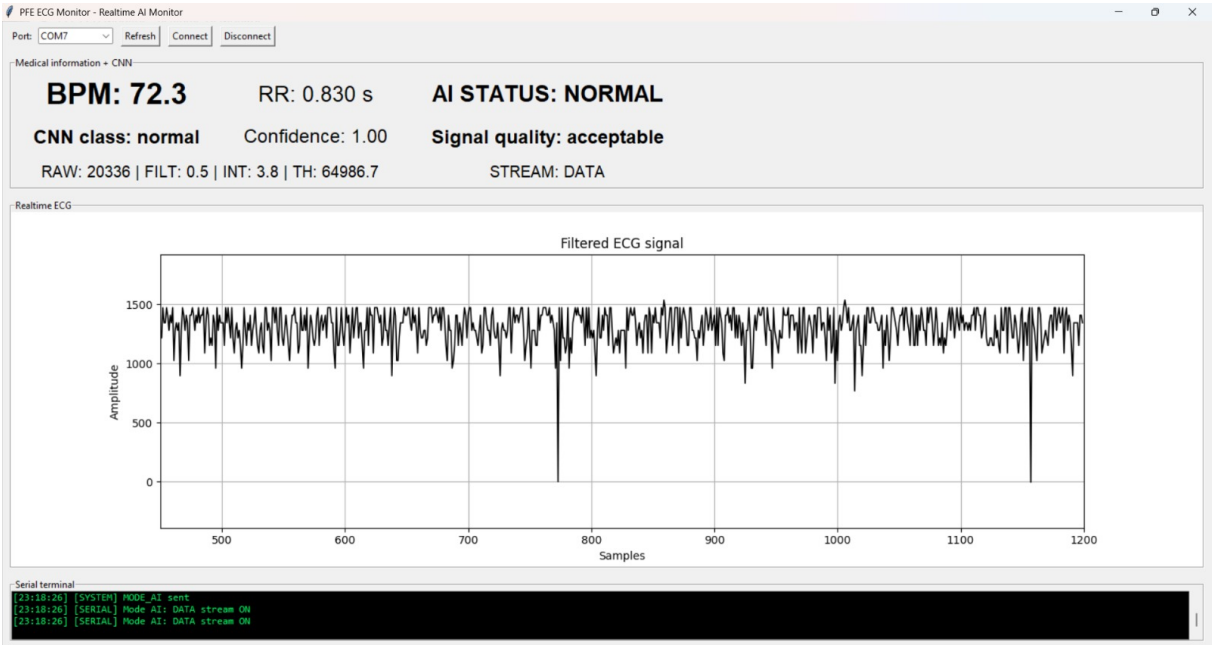


Figure 10: Real-time AI monitor showing ECG waveform and CNN classification.

# 4 Results and Discussion

## 4.1 System Operation

The developed prototype successfully performed:

- Real-time ECG acquisition;
- Analog-to-digital conversion;
- Embedded signal preprocessing;
- R-peak detection;
- BPM estimation;
- RR interval calculation;
- Serial data transmission;
- Real-time visualization;
- CNN-based classification on the computer.

## 4.2 Signal Quality

Signal quality was strongly influenced by electrode placement and contact stability. During the development phase, it was observed that poor electrode contact could produce false readings or unstable ECG waveforms. For this reason, the firmware and the graphical monitor were adjusted to detect abnormal conditions and display signal quality information.

The main sources of signal degradation were:

- Electrode displacement;
- Cable movement;
- Poor skin contact;
- Environmental electromagnetic noise;
- Baseline fluctuations.

The signal quality indicator was classified as good, acceptable, or bad according to the behavior of the acquired ECG waveform and its consistency over time.

## 4.3 CNN Performance

The CNN evaluation was performed using the dataset generated for the project. The test set contained 4601 ECG windows.

The model output order did not match the alphabetical class order. Therefore, an inferred mapping was used to correctly associate each model output neuron with its corresponding ECG class.

The inferred mapping was:

Table 1: Inferred mapping between CNN output neurons and ECG classes.

| Model output | ECG class |
|---|---|
| Output 0 | normal |
| Output 1 | supraventricular |
| Output 2 | ventricular |
| Output 3 | fusion |
| Output 4 | other |

Using this mapping, the final accuracy obtained on the test set was:

$$Accuracy = 0.9063 \tag{21}$$

or approximately:

$$Accuracy = 90.63\% \tag{22}$$

The detailed classification metrics are presented in Table 2.

Table 2: CNN classification performance on the test set.

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| fusion | 0.000 | 0.000 | 0.000 | 14 |
| normal | 0.895 | 0.984 | 0.937 | 3147 |
| other | 0.000 | 0.000 | 0.000 | 123 |
| supraventricular | 0.964 | 0.509 | 0.667 | 318 |
| ventricular | 0.937 | 0.913 | 0.925 | 999 |
| Accuracy | | 0.9063 | | 4601 |
| Macro Avg | 0.559 | 0.481 | 0.506 | 4601 |
| Weighted Avg | 0.878 | 0.906 | 0.887 | 4601 |

The results show that the CNN performed well for the normal and ventricular classes. The normal class obtained an F1-score of 0.937, while the ventricular class obtained an F1-score of 0.925.

The supraventricular class presented high precision but lower recall. This indicates that, when the model predicted this class, it was usually correct, but it missed a significant number of true supraventricular examples.

The fusion and other classes were not correctly classified in this evaluation. This behavior is probably related to the limited number of examples in these classes and class imbalance in the dataset. For example, the fusion class had only 14 samples in the test set, while the normal class had 3147 samples.

## 4.4 AI Reliability Analysis

The reliability of the proposed artificial intelligence system was evaluated by analyzing the classification report, the confusion matrix, the normalized confusion matrix, the prediction confidence distribution, and example ECG windows generated from the test set.

The global accuracy of 90.63% indicates that the CNN was able to correctly classify most ECG windows in the test set. However, the class-wise analysis shows that the model performance was not uniform across all classes.

The normal and ventricular classes were the most reliable. This is important because these classes were also the most relevant during real-time monitoring tests. The AI monitor was able to classify normal ECG windows in real time while displaying BPM, RR interval, signal quality, and confidence.

The results also showed that class imbalance affected the CNN behavior. Classes with fewer samples, such as fusion and other, obtained poor recall and F1-score. This suggests that future training should include dataset balancing, data augmentation, and possibly class-weighted loss functions.

The precision, recall, and F1-score are defined as:

$$Precision = \frac{TP}{TP + FP} \tag{23}$$

$$Recall = \frac{TP}{TP + FN} \tag{24}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{25}$$

where $TP$ represents true positives, $FP$ false positives, and $FN$ false negatives. The CNN evaluation figures are shown below.

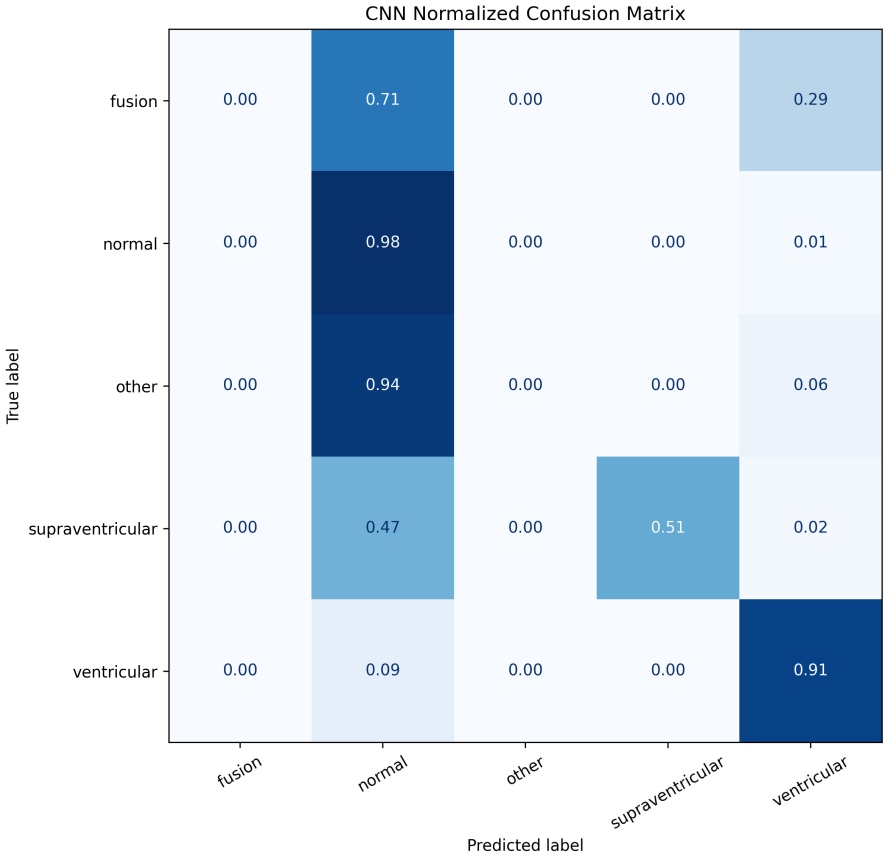


Figure 11: CNN confusion matrix obtained from the test set.

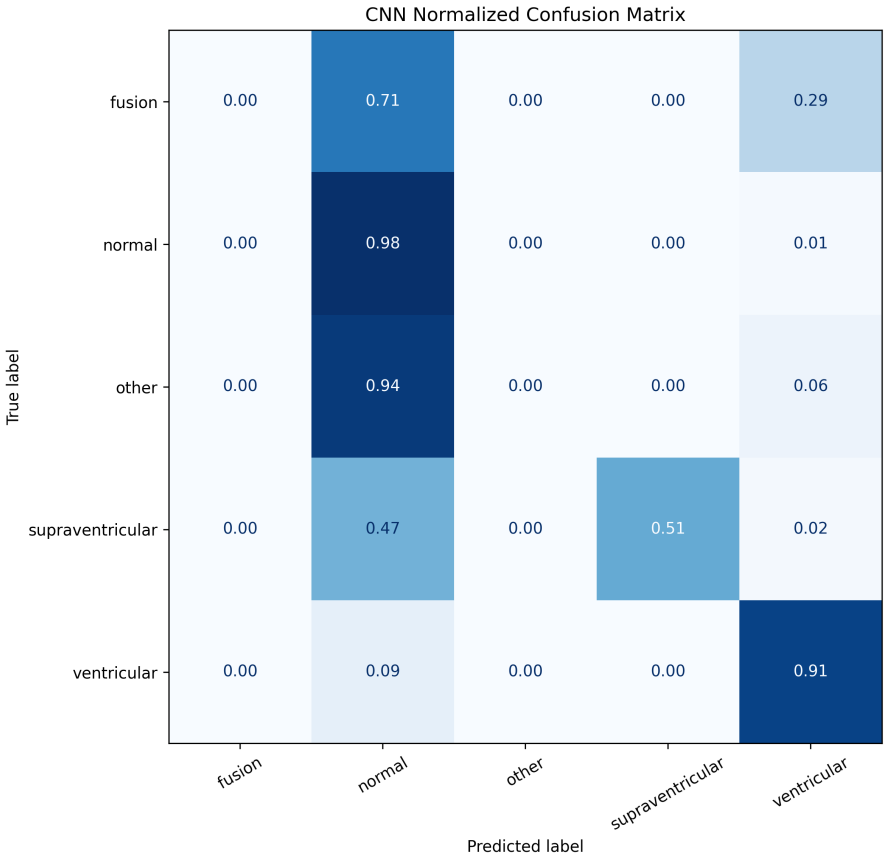


Figure 12: Normalized CNN confusion matrix.

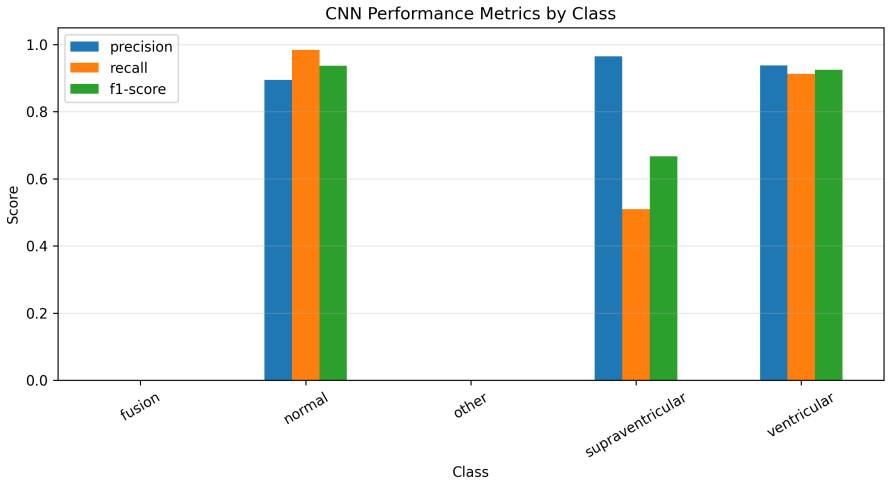


Figure 13: Precision, recall, and F1-score by ECG class.

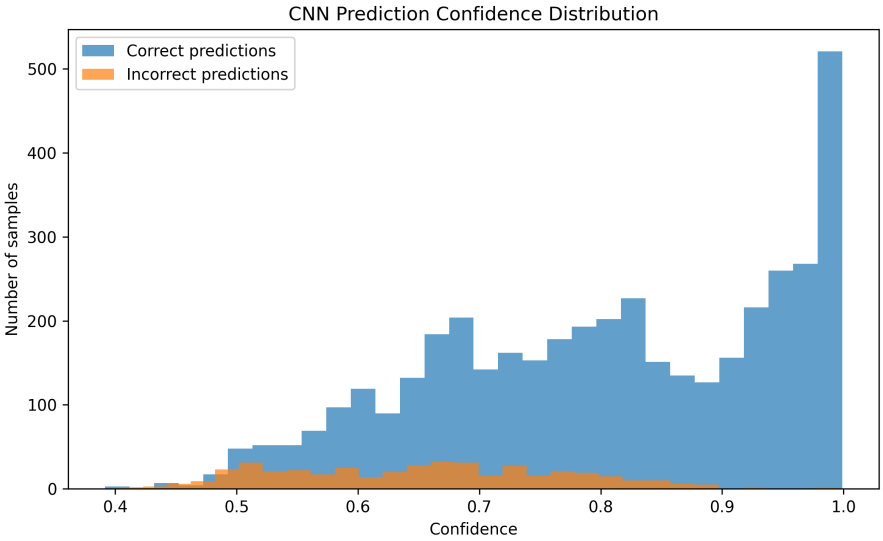


Figure 14: CNN prediction confidence distribution.

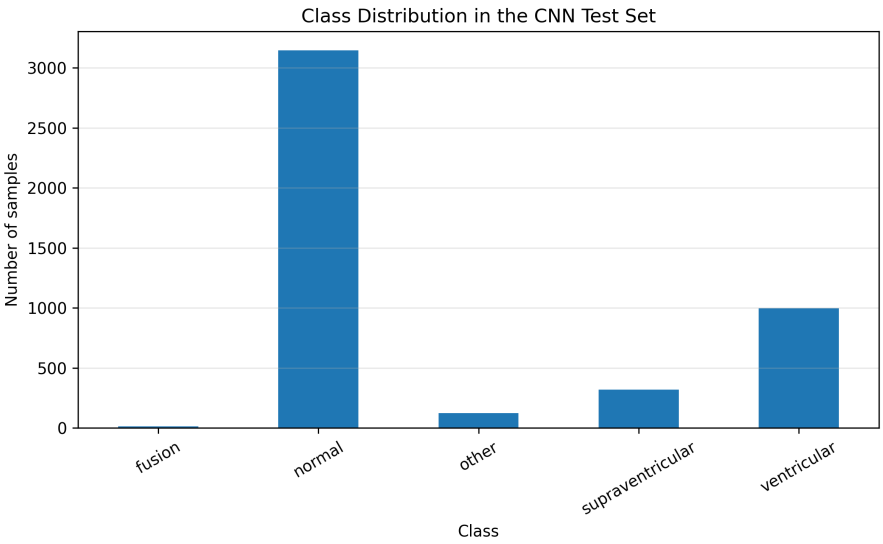


Figure 15: Class distribution in the CNN test set.

Examples of ECG windows used in the CNN evaluation are presented in Figures **??**, **??**, and **??**.

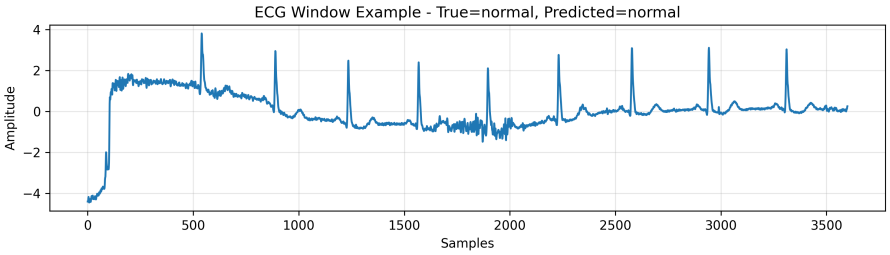


Figure 16: Example ECG window classified as normal.

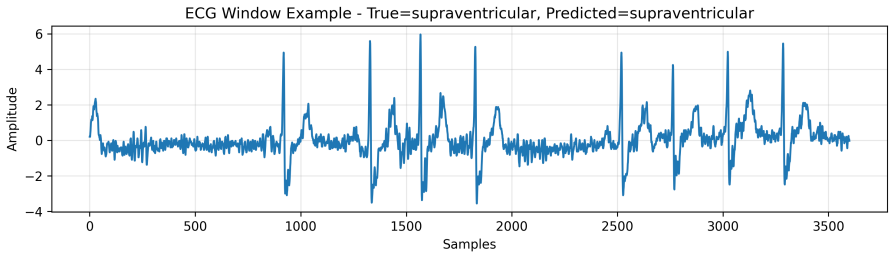


Figure 17: Example ECG window classified as supraventricular.

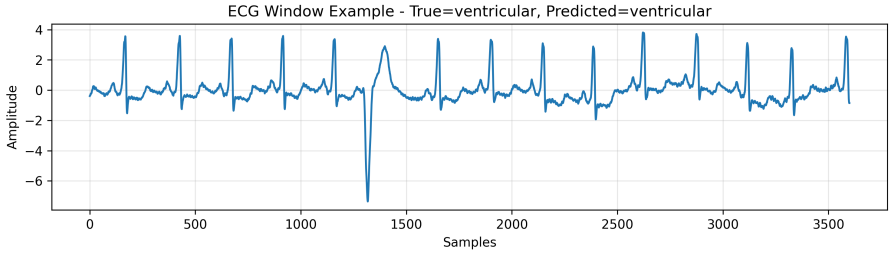


Figure 18: Enter Caption

Overall, the AI analysis demonstrated that the proposed CNN is suitable for proof-of-concept ECG classification, especially for normal and ventricular patterns. However, the model still requires improvement before being considered reliable for clinical use.

## 4.5 Discussion

The obtained results demonstrate the feasibility of combining embedded ECG acquisition with real-time AI-assisted analysis. The ESP32-S3 successfully acquired ECG-related data and transmitted it to the monitoring interface. The Python application displayed the waveform, BPM, RR interval, signal quality, and CNN predictions.

The system is especially relevant as a low-cost educational and research platform. It allows visualization of the complete ECG acquisition chain, from the electrodes to the AI-based classification interface.

However, the results also highlight important limitations. ECG acquisition is highly sensitive to noise and electrode placement. In addition, CNN reliability depends strongly on the quality and balance of the training dataset. Therefore, the proposed system should be interpreted as a functional prototype and not as a clinically validated diagnostic device.

# 5 Project Management and Gantt Diagram

The project was developed during the current month, from the beginning of the month until the present report submission period. Due to the short development time, the work was organized in weekly phases instead of monthly phases.

Table 3: Planned and performed project schedule during the development month.

| Task | Week 1 | Week 2 | Week 3 | Week 4 |
|---|---|---|---|---|
| Literature review | X | X | | |
| Hardware selection | X | | | |
| Prototype assembly | X | X | | |
| Firmware development | | X | X | |
| ECG signal processing | | X | X | X |
| R-peak and BPM validation | | | X | X |
| CNN training and evaluation | | | X | X |
| Graphical interface | | X | X | X |
| System testing and debugging | | | X | X |
| Report writing | | | X | X |

The real execution was concentrated in a short period of approximately one month. The initial phase focused on hardware selection and prototype assembly. The intermediate phase involved firmware development, ECG signal acquisition, filtering, and R-peak detection. The final phase focused on CNN integration, graphical interface development, system testing, analysis of results, and report writing.

Some tasks required more time than initially expected, especially hardware debugging, electrode contact stabilization, serial communication, and the integration between the ESP32-S3 firmware and the Python monitoring interface.

# 6 Ecological Transition and Sustainable Development

The proposed project is aligned with sustainability principles because it aims to develop a low-cost and reusable biomedical monitoring platform.

From an ecological and economic point of view, the use of affordable components such as the ESP32-S3, AD8232, and ADS1115 reduces the cost of prototyping and allows easier replication in educational and research environments.

The project also contributes to sustainable development by promoting:

- Low-cost healthcare technology;
- Reusable embedded hardware;
- Reduced dependence on expensive proprietary equipment;
- Open and reproducible development;
- Potential support for remote monitoring and telemedicine.

Although the prototype is not yet a certified medical device, it demonstrates how accessible electronic platforms can be used to support health-related innovation.

Future versions could further improve sustainability by using rechargeable batteries, optimized power consumption, recyclable enclosures, and wireless communication to reduce cable usage.

# 7 Limitations

The proposed system presents several limitations:

- The system was not clinically validated;
- Only a limited number of ECG leads was used;
- The real ECG signal was affected by noise and electrode contact quality;
- The CNN dataset was imbalanced;
- Some classes had poor classification performance;
- AI inference was performed on the computer, not fully embedded in the ESP32-S3;
- The prototype is intended for research and educational purposes only.

These limitations are important because they define the current scope of the project. The system should not be interpreted as a replacement for medical ECG equipment or professional diagnosis.

# 8 Conclusion

## 8.1 General Conclusion

This work presented the development of an intelligent real-time ECG monitoring system based on low-cost embedded hardware and artificial intelligence.

The developed prototype successfully integrated ECG acquisition, analog-to-digital conversion, embedded processing, R-peak detection, BPM calculation, serial communication, graphical visualization, and CNN-based classification.

The ESP32-S3, AD8232, and ADS1115 formed a functional acquisition platform. The Python interface allowed real-time visualization of cardiac information and AI-based classification.

The CNN evaluation obtained an overall accuracy of 90.63% on the test set when using the inferred output-class mapping. The model performed well for normal and ventricular classes, but showed limitations for minority classes such as fusion and other.

Therefore, the project achieved its main objective as a proof-of-concept intelligent ECG monitoring platform.

## 8.2 Future Work

Future improvements may include:

- Development of a PCB;
- Improving the analog front-end and electrode connection stability;
- Using better filtering techniques for noise reduction;
- Increasing the number of real ECG acquisitions;

- Balancing the CNN dataset;
- Improving classification of minority classes;
- Deploying TinyML inference directly on the ESP32-S3;
- Using MQTT, BLE, or WiFi communication;
- Developing a mobile application;
- Performing validation with controlled biomedical datasets;
- Investigating clinical validation in partnership with healthcare professionals.

# References


[1] Pan, J.; Tompkins, W. J. A Real-Time QRS Detection Algorithm. *IEEE Transactions on Biomedical Engineering*, 1985.

[2] Moody, G. B.; Mark, R. G. The MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 2001.

[3] Kiranyaz, S.; Ince, T.; Gabbouj, M. Real-Time Patient-Specific ECG Classification by 1-D Convolutional Neural Networks. *IEEE Transactions on Biomedical Engineering*, 2016.

[4] Zhang, D. et al. Interpretable Deep Learning for Automatic Diagnosis of 12-Lead Electrocardiogram. *iScience*, 2021.

[5] Analog Devices. AD8232 Heart Rate Monitor Front End Datasheet.

[6] Texas Instruments. ADS1115 Ultra-Small, Low-Power, 16-Bit Analog-to-Digital Converter Datasheet.

[7] Espressif Systems. ESP32-S3 Technical Reference Manual.

[8] TensorFlow. TensorFlow Documentation. Available at: `https://www.tensorflow.org/`.

[9] Scikit-learn. Machine Learning in Python. Available at: `https://scikit-learn.org/`.